

PacTrain: Pruning and Adaptive Sparse Gradient Compression for Efficient Collective Communication in Distributed Deep Learning

Yisu Wang, Ruilong Wu, Xinjiao Li, Dirk Kutscher*

The Hong Kong University of Science and Technology (Guangzhou)

* Corresponding author is Dirk Kutscher, dku@hkust-gz.edu.cn

Abstract—Large-scale deep neural networks (DNN) exhibit excellent performance for various tasks. As DNNs and datasets grow, distributed training becomes extremely time-consuming and demands larger clusters. A main bottleneck is the resulting gradient aggregation overhead. While gradient compression and sparse collective communication techniques are commonly employed to alleviate network load, many gradient compression schemes do not achieve acceleration of the training process while also preserving accuracy. This paper introduces PacTrain, a novel framework that accelerates distributed training by combining pruning with sparse gradient compression. Active pruning of the neural network makes the model weights and gradients sparse. By ensuring the global knowledge of the gradient sparsity among all distributed training workers, we can perform lightweight compression communication without harming accuracy. We show that the PacTrain compression scheme achieves a near-optimal compression strategy while remaining compatible with the all-reduce primitive. Experimental evaluations show that PacTrain improves training throughput by 1.25 to $8.72\times$ compared to state-of-the-art compression-enabled systems for representative vision and language models training tasks under bandwidth-constrained conditions.

I. INTRODUCTION

Over the past decade, deep learning (DL) has seen significant growth driven by the development of increasingly complex large-scale models and massive datasets [1]–[3]. These DL models exhibit excellent performance in various computer vision (CV) and natural language processing (NLP) tasks [4]–[7]. However, their immense scale makes large-scale distributed training (DT) time-consuming and expensive [8].

Large-scale DT incurs a high communication overhead, which is a challenge that spans large and small-scale setups. In large clusters with tens of thousands of GPUs provided by cloud providers (e.g., AWS, Azure, Alibaba Cloud) [9], [10], communication overhead increases linearly as the number of workers increases. Similarly, non-hyperscaler deployments, such as single- or multi-compute clusters in university labs with consumer-grade GPUs, often face communication bottlenecks owing to the constrained and highly variable bandwidths in such systems [8].

Gradient compression (GC) has emerged as a promising solution for mitigating this communication bottleneck by reducing the volume of the exchanged gradient data [11]–[18]. Most compression schemes (e.g., TopK [15], TernGrad

[13], THC [14], and DGC [16]) are lossy and introduce some compression errors, meaning they often compromise both the convergence rate and attainable accuracy, and thus fail to accelerate the training process.

Another potential approach for addressing communication bottlenecks in DDL is sparse collective communication (SCC). In recent years, practitioners in the deep learning community have observed that gradient sparsity often occurs in DNN training and have sought to exploit it [19]–[22]. For instance, OmniReduce [19] introduces a streaming aggregation to transfer sparse gradient blocks, and Zen [20] presents a hashing algorithm that leverages parallel GPU computing to achieve balanced communications. However, these approaches assume that gradients are sparse, which may not always be the case in practice.

Nevertheless, as pointed out by prior studies [11], [18], a common problem of GC and SCC is that most schemes (e.g., DGC, OmniReduce, and Zen) are not compatible with all-reduce. All-reduce [23], [24] is a highly scalable collective communication operation used to aggregate gradients among a set of DT workers. Another widely used gradient collection method is the parameter server (PS) that performs gradient collection on a centralized server [25]. All-reduce is inherently more parallel and scalable than PS based aggregation [11].

To address these challenges, we present PacTrain, a communication-efficient distributed training framework that combines model pruning and adaptive gradient compression. PacTrain can actively change the training gradient distribution through an unstructured pruning technique and leverages an efficient, non-lossy gradient compression scheme that is compatible with the all-reduce primitive to exploit gradient sparsity.

However, transforming this high-level idea into an existing DT framework, such as PyTorch Distributed Data Parallel (DDP) [26] is not straightforward because of the abstracted communication interface. DDP is a widely used framework that accelerates training by distributing model replicas across multiple GPUs, synchronizing gradients, and averaging them to ensure consistency. Most DT frameworks, including DDP, offer communication hook functions that enable developers to customize gradient synchronization algorithms. However, these hooks typically expose only a reformatted one-

TABLE 1: Impact of acceleration methods on training metrics.

Method	Conv. Speed	Compatibility	TTA
PacTrain	✓	✓	✓
THC	✓	✗	✓
Terngrad	✗	✓	?
DGC	✗	✓	?
OmniReduce	✓	✗	✓
Zen	✓	✗	✓

Each method may positively (marked ✓) or negatively (marked ✗) affect performance. The performance of the methods marked with ? will vary according to the choice of the model architecture and training dataset. Compatibility denotes whether the method is compatible with all-reduce operation.

dimensional gradient tensor, where gradient names are removed and gradient orders are rearranged. This abstraction complicates the establishment of a direct correspondence between the reformatted gradient tensor and the original model weights. Re-implementing the distributed training mechanism from scratch risks losing the performance optimizations inherent to the native frameworks. To address this, we propose a mask tracker mechanism that captures sparsity patterns within the reformatted gradient tensor, enabling efficient alignment and customization.

By ensuring that all distributed training workers share global knowledge of gradient sparsity, we can use a pruning mask to reformat sparse gradients into low-dimensional dense gradient tensors. This approach enables lightweight gradient compression without compromising accuracy during the training process. Tab. 1 summarizes how the different methods of GC and SCC impact one or both factors that determine the Time-To-Accuracy (TTA): the iteration time and convergence speed compared with PacTrain. Compared to these methods, PacTrain can provide both fast convergence and compatibility with all-reduce, and it can improve TTA performance.

We conducted extensive experiments on two datasets: CIFAR-10, CIFAR-100 [27], using VGG19 [28], ResNet18, ResNet152 [29], and ViT-Base-16 [6] models. Our evaluations include specific link speeds of 100 Mbps, 500 Mbps, and 1 Gbps, chosen to reflect the WAN bandwidth in small multicloud scenarios. Our evaluation shows that PacTrain effectively minimizes TTA under bandwidth-constrained conditions. Our implementation demonstrates that PacTrain successfully adapts the compression scheme to prune-aware-distributed training scenarios and reduces TTA by up to $8.72\times$ compared with native PyTorch DDP with variable-constrained network bandwidth.

This study makes the following contributions.

Contribution 1. We propose that pruning can be used to enhance the gradient compression, thereby accelerating the process. Traditional pruning after training drops a subset of weights to accelerate inference and save memory size, but we show that the gradient sparsity enforcement (GSE) method based on pruning is crucial for realizing efficient distributed

deep learning.

Contribution 2. We propose that all distributed training workers share global knowledge of gradient sparsity and propose a highly efficient, non-lossy compression scheme based on this property that is compatible with all-reduce operations. Our framework can be easily integrated into existing DT frameworks with minimal effort.

Contribution 3. Our comprehensive experiments on large-scale DL training tasks demonstrate that PacTrain outperforms traditional GC methods and achieves state-of-the-art TTA performance.

The rest of this paper is structured as follows: Section II discusses related work. Section III describes the system design and how to change the gradient distribution via pruning, and Section IV provides a detailed evaluation of different machine learning models in a real-world network, and Section V concludes the paper with a summary of our insights and a perspective on future work.

II. RELATED WORK

A. Gradient Aggregation

Data parallelism is widely used for training large-scale DL models. In data-parallel DDL, each GPU worker holds a replica of the DNN model, and the training dataset is partitioned among the workers. Each GPU worker independently performs *forward and backward propagation* on its partition, computing gradients that are aggregated among GPUs.

To mitigate the communication tension with training distributed DDL, many GC and SCC algorithms have been proposed in the community. Techniques such as quantization [14], sparsification [16], and sparse collective communication [19], [20] compress gradients to reduce the amount of data transmitted. However, as previously mentioned, existing GC and SCC methods often lack compatibility with all-reduce collectives. The primary challenge lies in avoiding decompression and recompression at each worker, as this could introduce substantial computational overhead or, in some cases, accumulate compression-induced errors, potentially impacting the training process.

B. Neural Network Pruning

Neural network pruning is a sparsification technique that reduces the memory cost and computation load by removing less important weights. The importance of model weights can be evaluated using various criteria, such as the absolute value of the weights [30] and the L1/L2 norm of the layers [31]. Conventional pruning techniques are mainly applied in a train-prune-fine-tune manner [32] to improve efficiency at the inference stage.

Pruning-Aware-Training. Conventional exploration of pruning during the training reduced training computation cost at each iteration but suffered noticeable performance degradation. Recently, researchers have proposed several policies to prune as early as possible during training without hurting performance such as the Lottery Ticket Hypothesis (LTH) [33], [34] and Early Pruning Indicator [35].

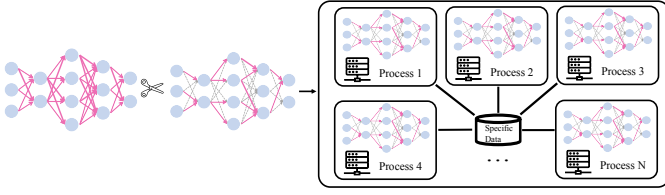


Fig. 1: Pruning and Fine-tune on specific data.

LTH shows that the existence of sparse sub-networks at initialization can achieve almost the same performance as the dense model when trained alone. By identifying these optimal sparse sub-networks after several training epochs, we can prune most of the model weights and reduce both the computation and communication costs during training.

The effectiveness of these strategies is also based on the empirical observation that network representations remain similar after the early phase of training [36]. Once the model stabilizes, only a small portion of the weights in the network is critical to its performance. Therefore, we can perform targeted training without the need to compute the gradients for all neurons [37].

Pruning-Aware-Distributed-Training. When we know in advance that the model weights are sparse, we can leverage this sparsity to customize our specific compression algorithm based on the sparse distribution of the model, which leads us to optimize the precision of the compression techniques and further reduce the gradient synchronization overhead.

Recent studies [38]–[42] accelerate distributed learning through dynamic sparse training. For example, DeepZero [38] employs pruning in zeroth-order optimization to induce gradient sparsity and improve training efficiency. FedMef [39], FedDST [41], and FedTiny [42] perform pruning and regrowth on devices to expedite federated learning (FL). However, pruning typically can achieve only 30% to 80% sparsity, which is insufficient for OmniReduce’s compression algorithm which requires approximately 1% sparsity. Using compression schemes like COO (Coordinate List) also increases heavy computational overhead. Unfortunately, these studies do not detail how to efficiently compress and transmit the sparse gradients.

III. PACTRAIN DESIGN

This section first outlines the problem setup and describes the system design of PacTrain. We then explain how pruning changes the gradient distribution and how the gradient binarization technique preserves the accuracy impacted by pruning.

A. Problem Setup

Researchers typically start with a model that has been pre-trained on a general-purpose dataset such as ImageNet [43]. Then they will re-train this model on a domain-specific dataset to meet the requirements of a specific task. To speed up training, the dataset is distributed across GPUs, with DDP used to enhance training efficiency. However, due to network

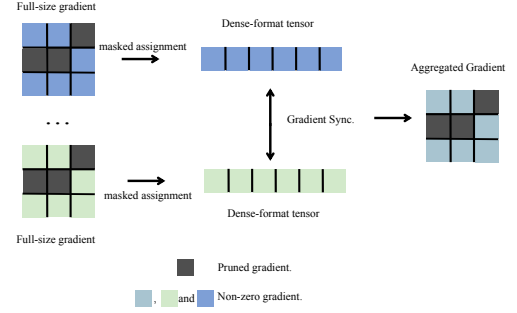


Fig. 2: Acceleration with masked assignment.

bottlenecks, gradient aggregation consumes a significant portion of the total training time.

Formally, we consider gradient aggregation phase as:

$$X^{t+1} = X^t - \eta \cdot \frac{1}{n} \sum_{i=1}^n \Delta(X^t, D_i^t) \quad (1)$$

where X^t represents the model weights at iteration t , n is the number of training workers, η represents the learning rate, D_i^t denotes the dataset partition assigned to the i -th GPU, and $\Delta(X^t, D_i^t)$ are the gradients computed by the i -th GPU based on its local data. Gradient aggregation ensures all GPU workers have the same updated model weights X^{t+1} for the next iteration.

Our objective is to derive a specialized sparse model with a mask m during the training process, aiming to optimize the TTA in DDP. Pruning a pre-trained model can lead to some accuracy loss, which depends on the pruning amount. The challenge in our optimization is how to enhance the communication speed in the gradient aggregation phase while maintaining the final test accuracy.

B. Design Overview

The design of PacTrain is motivated by the limitations of re-training and post-finetuning large models within bottleneck network link scenarios. Previous gradient sparsification techniques [15], [16] have two main limitations: computational overhead and incompatibility with all-reduce operations, limiting efficient gradient synchronization [11].

PacTrain provides less computational overhead and is compatible with all-reduce to preserve accuracy. To that end, we first train the model on a generic, smaller dataset like CIFAR-10 or TinyImageNet [44], or use an existing pre-trained model. The pre-trained model contains important weight information for the task, allowing us to perform pruning on the large model, as shown in Fig. 1. We then conduct distributed training on our specific dataset to accelerate convergence.

Fig. 2 shows how PacTrain implements a local sparsification technique. Instead of selecting TopK gradient coordinates, PacTrain selects non-zero gradient elements based on the local weight magnitudes, which remain the same across the distributed workers. The sparse gradients are reformatted into dense tensors to enable all-reduce synchronization, achieving

significant compression while maintaining computational efficiency.

C. Adaptive Compression Based on Pruning-Aware-Distributed-Training

As illustrated in Fig.1 and Fig.2, the design of PacTrain involves: a pre-trained model is provided to each distributed worker, which then performs task-specific fine-tuning on its subset of data. An unstructured pruning step ensures that the model weights are sparse, optimizing memory and computation efficiency. Subsequently, each worker engages in model update aggregation through the all-reduce operation.

Gradient Sparsity Enforcement (GSE). To reduce non-zero values in the neural network tensor, a pruning operation is necessary. However, during training, these weights can be updated to non-zero values through non-zero gradients. In other words, the distribution of the gradients is different from the distribution of the model weights. We can explicitly set the corresponding gradients to zero for model weights that are zero during the optimization process. This method ensures gradient sparsity. Specifically, this can be expressed using the following GSE formula:

$$\text{Gradient} = (\text{Weight} \neq 0) \odot \text{Gradient} \quad (2)$$

where $\odot$ denotes element-wise multiplication. By repeating this process in every training epoch, the model can progressively adapt to the task while maintaining synchronization in distributed environments.

Our algorithm leverages both sparse gradients and gradient compression during synchronization to significantly reduce communication and computation overhead. Unlike methods that rely solely on gradient magnitudes for compression, this algorithm incorporates the importance of model weights. It effectively eliminates low-magnitude gradients and aligns the gradient update process with both the pruning strategy and intrinsic gradient properties. This design improves data efficiency in constrained networks, accelerates convergence, and enhances the robustness of the model.

The detailed steps of the algorithm are as follows:

- 1) **Initialization:** Start with a pre-trained model and apply pruning using the specified pruning ratio to create a sparse network.
- 2) **Training:** During each training epoch, apply GSE to ensure that only gradients corresponding to non-zero weights are updated.
- 3) **Sparse Mask Update:** Use a mask tracker to monitor the distribution of sparse weights. When the mask stabilizes, compress the gradient updates to include only non-zero elements; otherwise, perform full synchronization to ensure correctness.
- 4) **Synchronization:** In the distributed environment, communicate only the non-zero elements of the sparse tensor to reduce communication overhead. For unstable masks, use full synchronization to maintain accuracy.

Algorithm 1: Worker Algorithms: Pruning Neural Networks and Send Sparse Tensor during AllReduce Operation

Data: Pre-trained model: The initial neural network model,

Number of epochs n : Total number of training epochs,

pruning_ratio : Ratio used for pruning the model,

Configuration : Configuration settings for distributed training,

Result: Trained sparse model

```

1 Initialize  $\text{Configuration}$  to find peers and aggregator;
2 Prune(model,  $\text{pruning\_ratio}$ ) ;
3 for  $i = 0$  to  $n$  epochs do
4   Perform training on the model;
5   Apply GSE to the model gradient;
6   foreach  $\text{update} \in \text{model\_updates}$  do
7     Update sparse mask using Mask Tracker;
8     if mask is stable then
9        $\text{compressed\_update} \leftarrow$ 
10         $\text{compress}(\text{update})$ ;
11        Communicate only non-zero elements in
12         $\text{compressed\_update}$ ;
8     else
9       Perform All-reduce on  $\text{update}$ ;
```

Mask Tracker mechanism. For each training epoch, the worker performs training on the sparse model. We introduce Mask Tracker (MT) to capture the sparsity patterns of the reformatted one-dimensional gradient tensor by maintaining the mapping of the gradient and the model weights. Existing DDP frameworks abstract the communication interface to the bucket and gradients are transformed into one-dimensional tensors during synchronization. This transformation removes the original names and order information of the gradients, making it challenging to establish a correspondence between the reformatted gradient tensors and the model weights.

The Mask Tracker monitors the sparsity patterns of the gradients without the need to redesign the underlying distributed training mechanism. By comparing this mask with the one from the previous iteration, we detect any changes in the sparsity pattern. If the mask remains unchanged over multiple iterations, MT deems the sparsity pattern stable. Once a pattern is considered stable, we communicate only the non-zero (non-masked) portions of the gradient tensor during synchronization. Because the MT operates directly on the flattened gradient tensors without modifying their values, it remains fully compatible with native floating-point AllReduce operations. This compatibility allows us to utilize existing optimizations and communication protocols without alteration.

D. Pruning with Gradient Quantization

To further reduce communication overhead while preserving final test accuracy, we introduce ternary gradient quantization

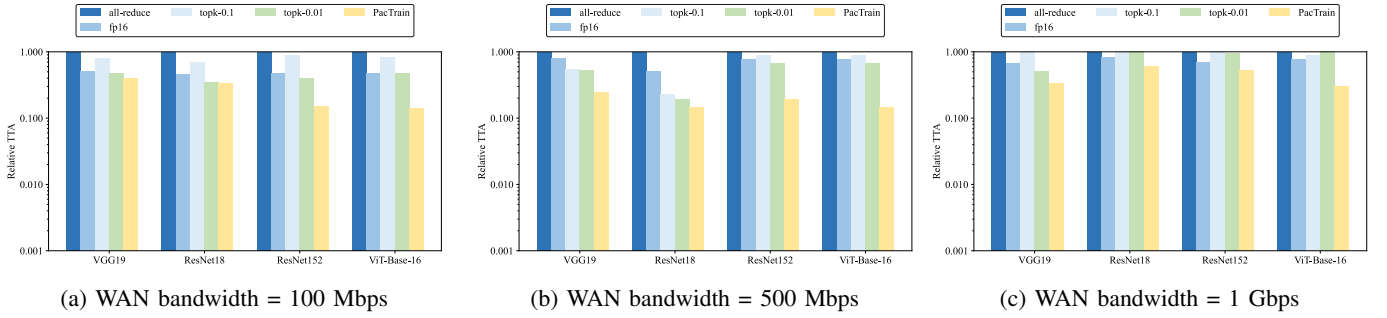


Fig. 3: End-to-End TTA speedup with different WAN bandwidths (relative to native all-reduce, log scale).

on top of importance-based pruning during DT. Our insight is applying Terngrad quantization to a pruned model can still direct optimization toward a desirable solution, as the pruning step already retains only the most important weight parameters. Specifically, in PacTrain, we use *ternarize*($\cdot$) to randomly quantize the gradient g_t into a ternary vector with values $\in \{-1, 0, +1\}$.

Pruning establishes a fixed mask that permanently zeros out gradients for less important parameters, thus confining the optimization process to a reduced parameter subspace. Although this can potentially slow down convergence or yield suboptimal solutions in non-convex neural networks, ternary quantization introduces controlled randomness. This stochastic element aids the model in escaping local minima.

Convergence Analysis. For each gradient component, the expected value of the quantized gradient is:

$$E(\tilde{g}_t) = E[s \cdot \text{sign}(g_t)] = s \cdot E(\text{sign}(g_t)) \quad (3)$$

where s is a scaling factor ensuring that the quantized gradient remains unbiased in expectation. However, quantization increases variance, which may cause fluctuations in the optimization direction and impact convergence speed.

We employ GraSP [45] to compute pruning scores that reflect each parameter’s contribution to the gradient flow. Specifically, the pruning score S is defined as:

$$S = -\theta \odot (H \nabla l(\theta)) \quad (4)$$

where $\nabla l(\theta)$ is the gradient of the loss function, and H is its Hessian matrix. This score allows us to retain parameters critical for maintaining essential gradient directions, thereby enhancing robustness.

In practice, determining the pruning mask based on weight ranking reduces sensitivity to NMSE (Normalized Mean Squared Error, $\text{NMSE}(x, \hat{x}) = \frac{\|x - \hat{x}\|^2}{\|x\|^2}$), further improving robustness in distributed training. This ensures that the remaining parameters retain the key gradients necessary for stable optimization.

IV. EVALUATION

PacTrain is designed to operate in arbitrary network settings, ranging from datacenter networks to the public Internet. We

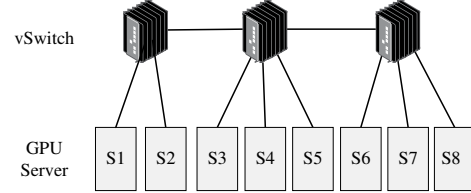


Fig. 4: Evaluation topology.

implemented and evaluated PacTrain using a virtual testbed hosted in our laboratory.

A. Setup and workloads

We built our test bed using the ESXI virtualization platform with a server equipped with eight A40 GPUs and an Intel Xeon Platinum 8358P processor with 64 CPU cores. To simulate the network environment shown in Fig. 4, we added three virtual switches, each connected to virtual machines with GPUs.

We developed the PacTrain prototype on top of the PyTorch distributed framework [26], implementing distributed machine learning communication, where each GPU performs a learning task and communicates with others using NCCL [46] over TCP for gradient aggregation. In our implementation, we utilized PyTorch’s *DistributedDataParallel* communication hook to override the default all-reduce operation, allowing fine-grained control over how gradients are communicated across workers.

Workloads. We evaluated our PacTrain prototype by training popular computer vision models on an eight-worker testbed. The models and datasets used in our evaluation included VGG19 [28], ResNet18, ResNet152 [29] and ViT-Base-16 [6], all of which were trained on the CIFAR-100 and CIFAR-10 dataset for 100 epochs. Unless otherwise noted, we set the per-GPU batch size to 32.

Metrics. We evaluate time-to-accuracy (TTA) across all training tasks under varying bandwidth conditions. We define convergence time as the duration required for the model to reach a target accuracy threshold, signifying that the model has fully converged.

B. Evaluation Scenarios

We conducted experiments under three different scenarios using the topology shown in Fig. 4, creating network perfor-

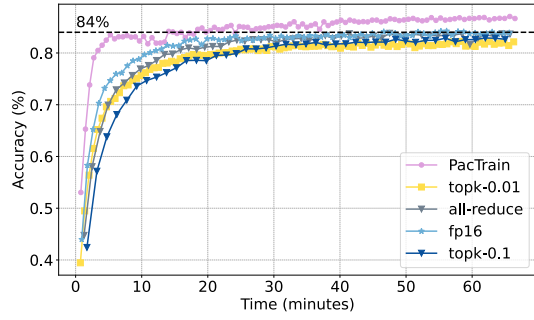


Fig. 5: Time-to-accuracy Comparison of performance for the CIFAR-10 classification task using ResNet152.

mance bottlenecks by adjusting the link bandwidths of the two connections to the switch.

We configured different bottleneck bandwidths of 100 Mbps, 500 Mbps, and 1 Gbps, representing typical network conditions in cloud environments and small datacenters.

C. End-to-End Performance

1) *Time-to-accuracy*: We evaluated PacTrain against FP16 quantization, TopK compression (0.1 rate and 0.01), and AllReduce for VGG19, ResNet18, ResNet152, and Vit-Base-16 models in bandwidth-constrained environments (100 Mbps, 500 Mbps, and 1 Gbps). As most gradient compression algorithms perform similarly to FP16 [11], we selected FP16 and the commonly used TopK algorithms for comparison. Under extremely low bandwidth conditions, PacTrain demonstrates superior convergence speed compared to FP16 and TopK. Unless otherwise specified, we use a compression scheme with a pruning ratio of 0.5.

Fig. 3 shows the end-to-end training time required to reach the same target accuracy in multiple bandwidth-constrained environments, normalized to that of the native all-reduce operation. The experimental results show that, even under constrained network bandwidth, PacTrain can continue training effectively, leading to faster convergence. Compared with methods that do not involve compression, PacTrain achieves a higher TTA reduction by up to $8.72\times$. Compared to solutions that support gradient compression and quantization (FP16 and TopK), PacTrain achieves TTA reductions of $1.25\text{--}7.05\times$.

TopK's gradient exchange relies on all-gather, causing network congestion at a 0.1 compression rate, while a 0.01 rate results in too much information loss, making convergence difficult.

2) *Iteration and convergence speed*: We compare ResNet152 on CIFAR-10 under a 1 Gbps bandwidth due to its representative slow convergence. Fig. 5 shows that for ResNet152, PacTrain reaches the 84% target accuracy $5.64\times$ faster than all-reduce baseline and $3.28\times$ faster than fp16. For models where gradient components are evenly distributed, such as ResNet152, PacTrain tends to achieve a better convergence rate due to its utilization of pruning and quantization strategies.

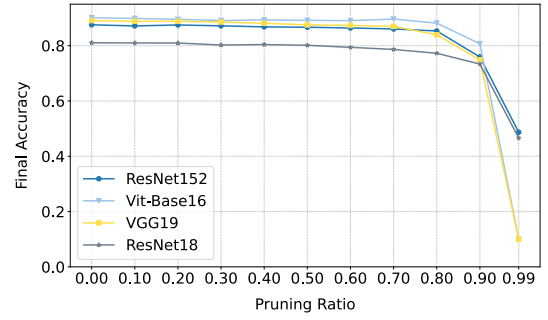


Fig. 6: Impact of Pruning Ratio on Final Accuracy for CIFAR-10 classification task.

We observed that model compression algorithms like TopK, which are incompatible with all-reduce, can slow convergence under good network conditions. This is because TopK requires more epochs to reach the target accuracy. When gradient aggregation is not a bottleneck, the benefits of compression are minimal, potentially hindering convergence. TopK relies on all-gather for gradient aggregation, which can lead to accuracy loss with low compression ratios or excessive communication overhead with high compression ratios in distributed training. In contrast, PacTrain, being compatible with all-reduce, ensures communication cost scales are proportional to the pruning ratio.

3) *Impact of Pruning Ratio on Final Accuracy*: Fig. 6 shows the impact of pruning ratios on final accuracy, based on our testing of VGG19, ResNet18, ResNet152, Vit-Base-16 models on the CIFAR-10 dataset. The relationship between the pruning ratio and final accuracy reflects a trade-off: as the pruning ratio increases, the final accuracy typically decreases due to the removal of important parameters. Although pruning introduces accuracy degradation, the impact remains minimal when the pruning ratio is kept below 80%. Taking ResNet152 as an example, as long as the pruning ratio does not exceed 80%, the final accuracy drops by less than 2%.

V. CONCLUSION

This paper presents PacTrain, a framework that leverages sparse communication to accelerate pruning-aware distributed learning, offering a novel solution to the communication bottlenecks faced in the distributed training of large models. To demonstrate generalizability, we conducted several experiments to demonstrate that our method can effectively reduce communication costs and maintain the model accuracy across various training scenarios. Testbed experiments using eight GPU workers under various bandwidth-constrained network conditions show that PacTrain can achieve up to an $8.72\times$ improvement.

VI. ACKNOWLEDGEMENTS

This work is supported by the Guangdong Provincial Project 2023QN10X048, the Guangzhou Key Laboratory on Future Networked Systems 2024A03J0623, and the Guangzhou Science and Technology Project 2023A03J0011.

REFERENCES

- [1] R. Thoppilan, D. D. Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, and et al., “Lmda: Language models for dialog applications,” 2022. [Online]. Available: <https://arxiv.org/abs/2201.08239>
- [2] H. Chen, Y. Miao, L. Chen, H. Sun, H. Xu, L. Liu, and et al., “Software-defined network assimilation: bridging the last mile towards centralized network configuration management with nassim,” ser. SIGCOMM ’22, 2022, p. 281–297.
- [3] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, “Netllm: Adapting large language models for networking,” ser. ACM SIGCOMM ’24, 2024, p. 661–678.
- [4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, and et al., “Llama: Open and efficient foundation language models,” 2023.
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, and et al., “Language models are few-shot learners,” ser. NIPS ’20. Curran Associates Inc., 2020.
- [6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, and et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” *ICLR*, 2021.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *CVPR*, 2016, pp. 779–788.
- [8] H. Lim, J. Ye, S. Abdu Jyothi, and D. Han, “Accelerating model training in multi-cluster environments with consumer-grade gpus,” ser. ACM SIGCOMM ’24, 2024, p. 707–720.
- [9] J. Cao, Y. Guan, K. Qian, J. Gao, W. Xiao, J. Dong, and et al., “Crux: Gpu-efficient communication scheduling for deep learning training,” ser. ACM SIGCOMM ’24. Association for Computing Machinery, 2024.
- [10] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, and et al., “MegaScale: Scaling large language model training to more than 10,000 GPUs,” in *NSDI* 24. USENIX Association, Apr. 2024, pp. 745–760.
- [11] W. Han, S. Vargaftik, M. Mitzenmacher, B. Karp, and R. B. Basat, “Beyond throughput and compression ratios: Towards high end-to-end utility of gradient compression,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01378>
- [12] R. Ben-Basat, Y. Ben-Itzhak, M. Mitzenmacher, and S. Vargaftik, “Optimal and near-optimal adaptive vector quantization,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.03158>
- [13] W. Wen, C. Xu, F. Yan, C. Wu, Y. Wang, Y. Chen, and H. Li, “Terngrad: Ternary gradients to reduce communication in distributed deep learning,” 2017. [Online]. Available: <https://arxiv.org/abs/1705.07878>
- [14] M. Li, R. B. Basat, S. Vargaftik, C. Lao, K. Xu, M. Mitzenmacher, and M. Yu, “Thc: Accelerating distributed deep learning using tensor homomorphic compression,” 2024. [Online]. Available: <https://arxiv.org/abs/2302.08545>
- [15] A. F. Aji and K. Heafield, “Sparse communication for distributed gradient descent,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, M. Palmer, R. Hwa, and S. Riedel, Eds. Association for Computational Linguistics, Sep. 2017, pp. 440–445.
- [16] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, “Deep gradient compression: Reducing the communication bandwidth for distributed training,” 2018, the full paper is available at <https://openreview.net/forum?id=rkxOQg-CZ>.
- [17] T. Vogels, S. P. Karimireddy, and M. Jaggi, “Powersgd: Practical low-rank gradient compression for distributed optimization,” 2020. [Online]. Available: <https://arxiv.org/abs/1905.13727>
- [18] H. Xu, C.-Y. Ho, A. M. Abdelmoniem, A. Dutta, E. H. Bergou, K. Karatsenidis, and et al., “Grace: A compressed communication framework for distributed machine learning,” in *ICDCS*, 2021.
- [19] J. Fei, C.-Y. Ho, A. N. Sahu, M. Canini, and A. Sapio, “Efficient sparse collective communication and its application to accelerate distributed deep learning,” ser. SIGCOMM ’21. Association for Computing Machinery, 2021, p. 676–691.
- [20] Z. Wang, Z. Xu, A. Shrivastava, and T. S. E. Ng, “Zen: Near-optimal sparse tensor synchronization for distributed dnn training,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.13254>
- [21] S. Li, Z. Lai, D. Li, Y. Zhang, X. Ye, and Y. Duan, “Embrace: Accelerating sparse communication for distributed training of deep neural networks,” ser. ICPP ’22. Association for Computing Machinery, 2023.
- [22] Z. Wang, H. Lin, Y. Zhu, and T. S. E. Ng, “Hi-speed dnn training with espresso: Unleashing the full potential of gradient compression with near-optimal usage strategies,” ser. EuroSys ’23. Association for Computing Machinery, 2023, p. 867–882.
- [23] Y. Wu, Y. Xu, J. Chen, Z. Wang, Y. Zhang, M. Lentz, and D. Zhuo, “Mccs: A service-based approach to collective communication for multi-tenant cloud,” ser. ACM SIGCOMM ’24. Association for Computing Machinery, 2024, p. 679–690.
- [24] D. D. Sensi, T. Bonato, D. Saam, and T. Hoeffer, “Swing: Short-cutting rings for higher bandwidth allreduce,” in *NSDI* 24. USENIX Association, Apr. 2024, pp. 1445–1462.
- [25] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, and et al., “Scaling distributed machine learning with the parameter server,” in *OSDI* 14. USENIX Association, Oct. 2014, pp. 583–598.
- [26] J. Ansel, E. Yang, H. He, N. Gimeshein, A. Jain, M. Voznesensky, and et al., “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” ser. ASPLOS ’24. Association for Computing Machinery, 2024, p. 929–947.
- [27] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep. 0, 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [28] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [30] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning both weights and connections for efficient neural networks,” 2015.
- [31] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient convnets,” 2017.
- [32] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, “Pruning convolutional neural networks for resource efficient inference,” 2017.
- [33] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” 2019.
- [34] X. Chen, Y. Cheng, S. Wang, Z. Gan, Z. Wang, and J. Liu, “Earlybert: Efficient bert training via early-bird lottery tickets,” 2021.
- [35] M. Shen, P. Molchanov, H. Yin, and J. M. Alvarez, “When to prune? a policy towards early structural pruning,” in *CVPR*, 2022.
- [36] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, “Similarity of neural network representations revisited,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.00414>
- [37] J. Jia, J. Liu, P. Ram, Y. Yao, G. Liu, Y. Liu, P. Sharma, and S. Liu, “Model sparsity can simplify machine unlearning,” 2024. [Online]. Available: <https://arxiv.org/abs/2304.04934>
- [38] A. Chen, Y. Zhang, J. Jia, J. Diffenderfer, J. Liu, K. Parasyris, Y. Zhang, Z. Zhang, B. Kailkhura, and S. Liu, “Deepzero: Scaling up zeroth-order optimization for deep model training,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.02025>
- [39] H. Huang, W. Zhuang, C. Chen, and L. Lyu, “Fedmef: Towards memory-efficient federated dynamic pruning,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.14737>
- [40] X. Qiu, J. Fernandez-Marques, P. P. Gusmao, Y. Gao, T. Parcollet, and N. D. Lane, “Zeroft: Efficient on-device training for federated learning with local sparsity,” 2022. [Online]. Available: <https://arxiv.org/abs/2208.02507>
- [41] S. Bibikar, H. Vikalo, Z. Wang, and X. Chen, “Federated dynamic sparse training: Computing less, communicating less, yet learning better,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.09824>
- [42] H. Huang, L. Zhang, C. Sun, R. Fang, X. Yuan, and D. Wu, “Distributed pruning towards tiny neural networks in federated learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.01977>
- [43] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “Imagenet large scale visual recognition challenge,” 2015. [Online]. Available: <https://arxiv.org/abs/1409.0575>
- [44] Y. Le and X. S. Yang, “Tiny imagenet visual recognition challenge,” 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16664790>
- [45] C. Wang, G. Zhang, and R. Grosse, “Picking winning tickets before training by preserving gradient flow,” 2020. [Online]. Available: <https://arxiv.org/abs/2002.07376>
- [46] NVIDIA, “Nvidia collective communications library (nccl),” <https://developer.nvidia.com/nccl>, 2024.